

UNIT-5

Unsupervised Machine Learning: Clustering basics, Types of Clustering Methods, K-Means Clustering, Agglomerative Hierarchical Clustering, DBSCAN Clustering, Grid-Based Clustering, DENCLUE Algorithm, Graph-Based Clustering, Minimum Spanning Tree (MST) Clustering, Scalable Clustering Algorithms, BIRCH Algorithm.

Clustering basics

Clustering is an **unsupervised learning** technique used to group similar data points into clusters based on their **similarity** or **distance**.

- The goal is to ensure that:
 - Data points **within the same cluster** are **similar** to each other.
 - Data points **from different clusters** are **dissimilar**

Types of Clustering Methods

There are **six main types** of clustering methods:

1. Partition-based Clustering
2. Hierarchical Clustering
3. Density-based Clustering
4. Grid-based Clustering
5. Model-based Clustering
6. Graph-based Clustering

1. Partition-based Clustering

- Divides the dataset into **non-overlapping clusters**.
- Each data point belongs to **exactly one cluster**.
- The goal is to **minimize intra-cluster distance** and **maximize inter-cluster distance**.

Examples:

- K-Means Clustering
- K-Medoids

➤ Advantages:

- Simple and efficient for large datasets.
- Works well with spherical cluster shapes.

➤ Disadvantages:

- Requires specifying the number of clusters (k).
- Sensitive to outliers and initialization.
- Struggles with non-spherical clusters.

2. Hierarchical Clustering

- Builds a **hierarchy (tree structure)** of clusters.
- Does **not require specifying the number of clusters** in advance.
- Represented using a **dendrogram**.

➤ Types:

1. **Agglomerative (Bottom-Up)**
 - Start with each point as a single cluster → merge until one cluster remains.
 - Example: **AGNES (Agglomerative Nesting)**
2. **Divisive (Top-Down)**
 - Start with one big cluster → recursively split into smaller clusters.
 - Example: **DIANA (Divisive Analysis)**

➤ Linkage Criteria:

- **Single Linkage:** Minimum distance between clusters.
- **Complete Linkage:** Maximum distance between clusters.
- **Average Linkage:** Average distance between all pairs of points.

➤ **Advantages:**

- Dendrogram gives visual insight.
- No need to predefine cluster count.

➤ **Disadvantages:**

- Computationally expensive for large datasets.
- Once merged/split, cannot be undone.

3. Density-based Clustering

- Clusters are formed based on **areas of high data density**.
- Points in dense regions are grouped; sparse regions are considered **noise or outliers**.

➤ **Examples:**

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
- OPTICS (Ordering Points To Identify Clustering Structure)
- DENCLUE

➤ **Key Parameters:**

- **ϵ (Epsilon):** Radius defining neighborhood.
- **MinPts:** Minimum number of points to form a dense region.

➤ **Advantages:**

- Detects **arbitrary-shaped clusters**.
- Handles **noise and outliers** well.

➤ **Disadvantages:**

- Difficult to determine ϵ and MinPts.
- Fails with varying densities.

4. Grid-based Clustering

- The data space is divided into a **finite number of grids (cells)**.
- Clusters are formed based on **dense cells** in the grid.

➤ **Examples:**

- STING (Statistical Information Grid)
- CLIQUE (CLustering In QUEst)
- WaveCluster

➤ **Advantages:**

- **Fast processing time** (depends on grid size, not number of data points).
- Efficient for **high-dimensional** data.

➤ **Disadvantages:**

- Sensitive to grid size.
- May produce poor clusters if grid resolution is not appropriate.

5. Model-based Clustering

- Assumes data is generated from a **mixture of underlying probabilistic models**.
- Each cluster corresponds to a **probability distribution** (e.g., Gaussian).

➤ **Examples:**

- **Gaussian Mixture Models (GMM)**
- **Expectation-Maximization (EM) Algorithm)**
- **Self-Organizing Maps (SOM)**

➤ **Advantages:**

- Provides a **statistical approach** to clustering.
- Can estimate **probability of membership** for each data point.

➤ **Disadvantages:**

- May get stuck in local optima.
- Assumes specific distribution shapes (e.g., Gaussian).

6. Graph-based Clustering

- Data points are treated as **nodes** in a graph.
- Edges represent similarity (or distance) between nodes.
- Clusters are formed by **cutting edges** in a way that minimizes within-cluster distance.

➤ **Examples:**

- **MST (Minimum Spanning Tree) Clustering**
- **Spectral Clustering**
- **CHAMELEON**

➤ **Advantages:**

- Works well for complex or non-convex clusters.
- Does not require specifying the number of clusters.

➤ **Disadvantages:**

- Computationally intensive for large graphs.
- Sensitive to similarity threshold.

K-MEANS CLUSTERING

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Step 1:

Initialization: Randomly we choose following two centroids (k=2) for two clusters.

In this case the 2 centroid are: $m_1=(1.0,1.0)$ and $m_2=(5.0,7.0)$.



Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Step 2:

- Thus, we obtain two clusters containing:
{1,2,3} and {4,5,6,7}.
- Their new centroids are:

$$m_1 = \left(\frac{1}{3}(1.0+1.5+3.0), \frac{1}{3}(1.0+2.0+4.0) \right) = (1.83, 2.33)$$

$$m_2 = \left(\frac{1}{4}(5.0+3.5+4.5+3.5), \frac{1}{4}(7.0+5.0+5.0+4.5) \right) = (4.12, 5.38)$$

Individual	Centroid 1	Centroid 2
1	0	7.21
2 (1.5, 2.0)	1.12	6.10
3	3.61	3.61
4	7.21	0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.92

$$d(m_1, 2) = \sqrt{|1.0-1.5|^2 + |1.0-2.0|^2} = 1.12$$

$$d(m_2, 2) = \sqrt{|5.0-1.5|^2 + |7.0-2.0|^2} = 6.10$$

Step 3:

- Now using these centroids we compute the Euclidean distance of each object, as shown in table.
- Therefore, the new clusters are:
 $\{1,2\}$ and $\{3,4,5,6,7\}$
- Next centroids are:
 $m1=(1.25,1.5)$ and $m2 = (3.9,5.1)$



Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

Step 4 :

- The clusters obtained are:
 $\{1,2\}$ and $\{3,4,5,6,7\}$

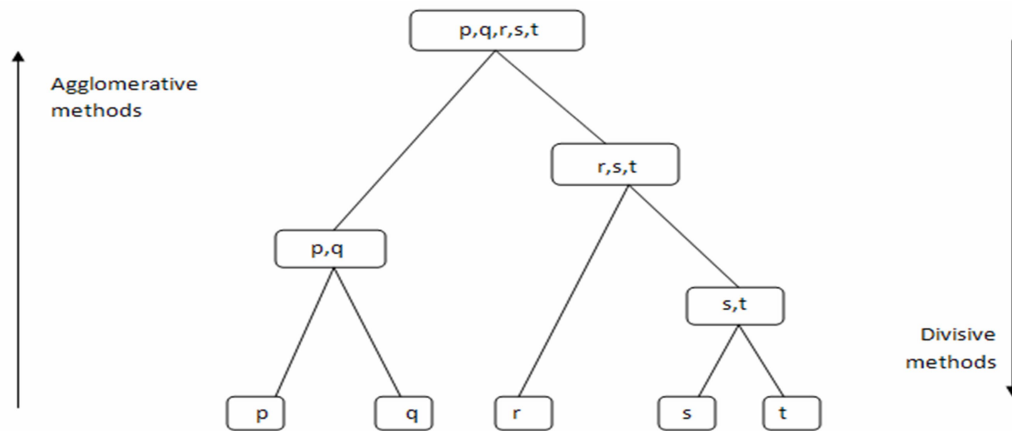
- Therefore, there is no change in the cluster.
- Thus, the algorithm comes to a halt here and final result consist of 2 clusters $\{1,2\}$ and $\{3,4,5,6,7\}$.



Individual	Centroid 1	Centroid 2
1	0.58	5.02
2	0.58	3.92
3	3.05	1.42
4	6.88	2.20
5	4.18	0.41
6	4.78	0.81
7	3.75	0.72

HIERARCHICAL CLUSTERING

1. Divisive hierarchical clustering
2. Agglomerative hierarchical clustering
 - a. Single linkage clustering
 - b. Complete linkage clustering
 - c. Average linkage clustering



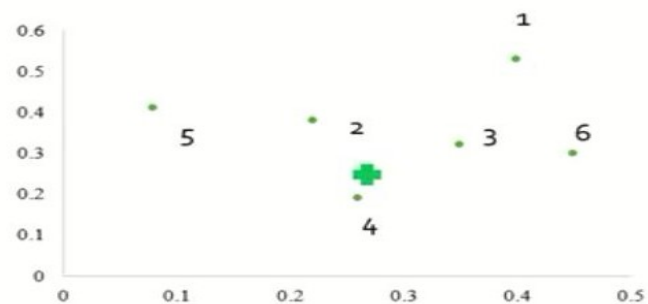
HIERARCHICAL AGGLOMERATIVE CLUSTERING

- Starts with one cluster , individual item in its own cluster and iteratively merge clusters until all the items belong to one cluster.
- Bottom up approach is followed to merge the clusters together.
- Dendrograms are pictorially used to represent the HAC.
- **S**ingle-nearest distance or single linkage.
- **C**omplete-farthest distance or complete linkage.
- **A**verage-average distance or average linkage.
- **A** tree like structure which represents hierarchical technique.
 - ✓ Leaf- Individual.
 - ✓ Root – One cluster.
- A cluster at level 1, is the merger of its child cluster at level $i + 1$.

- Find the clusters using single link technique. Use Euclidean distance, and draw the dendrogram.

	X	Y
P1	0.40	0.53
P2	0.22	0.38
P3	0.35	0.32
P4	0.26	0.19
P5	0.08	0.41
P6	0.45	0.30

	X	Y
P1	0.40	0.53
P2	0.22	0.38
P3	0.35	0.32
P4	0.26	0.19
P5	0.08	0.41
P6	0.45	0.30



- Calculate Euclidean distance, create the distance matrix.

$$\text{Distance} [(x,y), (a,b)] = \sqrt{(x - a)^2 + (y - b)^2}$$

$$\begin{aligned} \text{Distance (P1,P2)} &= \sqrt{(0.40 - 0.22)^2 + (0.53 - 0.38)^2} \\ (0.40,0.53), (0.22,0.38) &= \sqrt{(0.18)^2 + (0.15)^2} \\ &= \sqrt{0.0324 + 0.0225} \\ &= \sqrt{0.0549} \\ &= 0.23 \end{aligned}$$

- Calculate Euclidean distance, create the distance matrix.

$$\text{Distance } [(x,y), (a,b)] = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\begin{aligned} \text{Distance (P1,P2)} &= \sqrt{(0.40 - 0.22)^2 + (0.53 - 0.38)^2} \\ (0.40, 0.53), (0.22, 0.38) &= \sqrt{(0.18)^2 + (0.15)^2} \\ &= \sqrt{0.0324 + 0.0225} \\ &= \sqrt{0.0549} \\ &= 0.23 \end{aligned}$$

- The distance matrix is

	P1	P2	P3	P4	P5	P6
P1	0					
P2	0.23	0				
P3	0.22	0.15	0			
P4	0.37	0.20	0.15	0		
P5	0.34	0.14	0.28	0.29	0	
P6	0.23	0.25	0.11	0.22	0.39	0

- The distance matrix is

	P1	P2	P3	P4	P5	P6
P1	0					
P2	0.24	0				
P3	0.22	0.15	0			
P4	0.37	0.20	0.15	0		
P5	0.34	0.14	0.28	0.29	0	
P6	0.23	0.25	0.11	0.22	0.39	0

- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P3}}, \underline{\text{P6}}), \underline{\text{P1}})]$
- $\text{MIN}(\text{dist}(\text{P3}, \text{P1}), (\text{P6}, \text{P1}))$
 $= \min[(0.22, 0.23)]$
 $= 0.22$
- To update the distance matrix $\text{MIN}[\text{dist}(\text{P3}, \text{P6}), \text{P2}]$
- $\text{MIN}(\text{dist}(\text{P3}, \text{P2}), (\text{P6}, \text{P2}))$
 $= \min[(0.15, 0.25)]$
 $= 0.15$

- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P3}}, \underline{\text{P6}}), \underline{\text{P1}})]$
- $\text{MIN}(\text{dist}(\text{P3}, \text{P1}), (\text{P6}, \text{P1}))$
 $= \min[(0.22, 0.23)]$
 $= 0.22$
- To update the distance matrix $\text{MIN}[\text{dist}(\text{P3}, \text{P6}), \text{P2}]$
- $\text{MIN}(\text{dist}(\text{P3}, \text{P2}), (\text{P6}, \text{P2}))$
 $= \min[(0.15, 0.25)]$
 $= 0.15$

- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P3}}, \underline{\text{P6}}), \underline{\text{P4}})]$
- $\text{MIN}(\text{dist}(\underline{\text{P3}}, \underline{\text{P4}}), (\text{P6}, \text{P4}))$
 $= \min[(0.15, 0.22)]$
 $= 0.15$
- To update the distance matrix $\text{MIN}[\text{dist}(\text{P3}, \text{P6}), \text{P5}]$
- $\text{MIN}(\text{dist}(\text{P3}, \text{P5}), (\text{P6}, \text{P5}))$
 $= \min[(0.28, 0.39)]$
 $= 0.28$

- The updated distance matrix for cluster P3, P6

	P1	P2	P3,P6	P4	P5
P1	0				
P2	0.23	0			
✚ P3,P6	0.22	0.15	0		
P4	0.37	0.20	0.15	0	
P5	0.34	0.14	0.28	0.29	0

- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P2}}, \underline{\text{P5}}), \underline{\text{P1}})]$
- $\text{MIN}[\text{dist}(\underline{\text{P2}}, \underline{\text{P1}}), (\underline{\text{P5}}, \underline{\text{P1}})]$
 $= \min[(0.23, 0.34)]$
 $= 0.23$
- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P2}}, \underline{\text{P5}}), (\underline{\text{P3}}, \underline{\text{P6}})]$
- $\text{MIN}[\text{dist}(\underline{\text{P2}}, (\underline{\text{P3}}, \underline{\text{P6}})), (\underline{\text{P5}}, (\underline{\text{P3}}, \underline{\text{P6}}))]$
 $= \min[(0.15, 0.28)]$
 $= 0.15$
- To update the distance matrix $\text{MIN}[\text{dist}(\underline{\text{P2}}, \underline{\text{P5}}), \underline{\text{P4}})]$
- $\text{MIN}[\text{dist}(\underline{\text{P2}}, \underline{\text{P4}}), (\underline{\text{P5}}, \underline{\text{P4}})]$
 $= \min[(\underline{0.20}, \underline{0.29})]$
 $= \underline{0.20}$

- The updated distance matrix for cluster P2,P5

	P1	P2,P5	P3,P6	P4
P1	0			
P2,P5	0.23	0		
P3,P6	0.22	0.15	0	
P4	0.37	0.20	0.15	0

- To update the distance matrix $\text{MIN}[\text{dist}((P2,P5),(P3,P6)),P1]$

- $\text{MIN}[\text{dist}((P2,P5),P1), ((P3,P6),P1)]$

$$= \min[(0.23,0.22)]$$

$$= 0.22$$

- To update the distance matrix $\text{MIN}[\text{dist}((P2,P5),(P3,P6)),P4]$

- $\text{MIN}[\text{dist}((P2,P5),P4), ((P3,P6),P4)]$

$$= \min[(0.20,0.15)]$$

$$= 0.15$$

- To update the distance matrix $\text{MIN}[\text{dist}(P2,P5,P3,P6),P4]$

- $\text{MIN}[\text{dist}((P2,P5,P3,P6),P1), (P4,P1)]$

$$= \min[(0.22,0.37)]$$

$$= 0.22$$

- The updated distance matrix for cluster P2,P5,P3,P6,P4

	P1	P2,P5,P3,P6,P4
P1	0	
P2,P5,P3,P6,P4	0.22	0

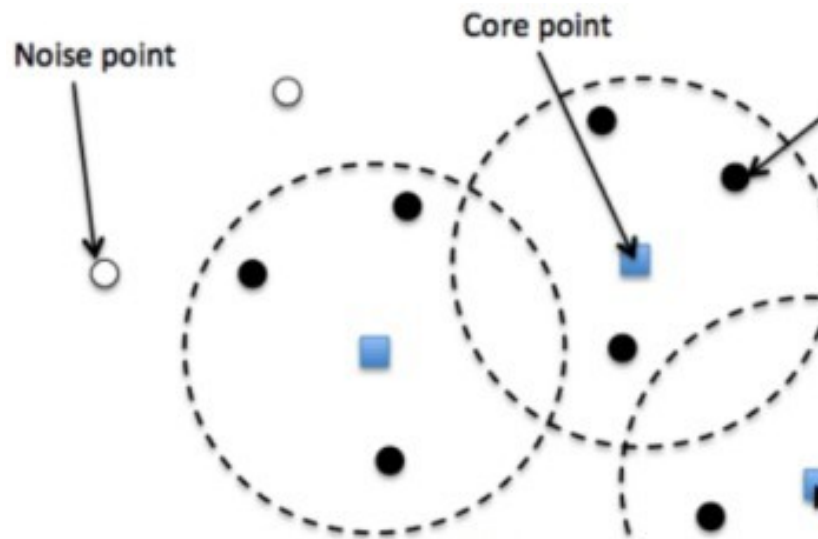
DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular learning method utilized in model building and machine learning algorithms. This is a clustering method that is used in machine learning to separate clusters of high density from clusters of low density. Clustering analysis or simply Clustering is an unsupervised learning method that divides the data points into several specific batches or groups, such that the data points in the same groups have similar properties and data points in different groups have different properties in some sense.

DBSCAN can also sort data into clusters of varying shapes, another strong advantage. DBSCAN works as such:

- Divide the dataset into n dimensions
- For each point in the dataset, DBSCAN forms an n-dimensional shape around that data point and then counts how many data points fall within that shape.
- DBSCAN counts this shape as a cluster. DBSCAN iteratively expands the cluster, by going through each individual point within the cluster and counting the number of other data points nearby.

Components of DBSCAN



The parameters of DBSCAN

The main idea behind DBSCAN is that a point belongs to a cluster if it is close to many points from that cluster. There are two key parameters of DBSCAN:

- **eps**: The distance that specifies the neighbourhoods. Two points are considered to be neighbours if the distance between them is less than or equal to eps.
- **minPts**: Minimum number of data points to define a cluster.

These parameters can be understood if we explore two concepts called Density Reachability and Density Connectivity.

- Reachability in terms of density establishes a point to be reachable from another if it lies within a particular distance (eps) from it.
- Connectivity, on the other hand, involves a transitivity-based chaining approach to determine whether points are located in a particular cluster. For example, p and q points could be connected if $p \rightarrow r \rightarrow s \rightarrow t \rightarrow q$, where $a \rightarrow b$ means b is in the neighbourhood of a.

The points of DBSCAN

Based on these two parameters, points are classified as core points, border points, or outliers:

- **Core point**: A point is a core point if there are at least minPts number of points (including the point itself) in its surrounding area with radius eps.

- **Border point:** A point is a border point if it is reachable from a core point and there are less than minPts number of points within its surrounding area.
- **Outlier** (also known as noise): A point is an outlier if it is not a core point and not reachable from any core points.

Steps Used In DBSCAN Algorithm

1. Find all the neighbor points within eps and identify the core points or visited with more than MinPts neighbors.
2. For each core point if it is not already assigned to a cluster, create a new cluster.
3. Find recursively all its density-connected points and assign them to the same cluster as the core point.
A point a and b are said to be density connected if there exists a point c which has a sufficient number of points in its neighbors and both points a and b are within the *eps distance*. This is a chaining process. So, if b is a neighbor of c , c is a neighbor of d , and d is a neighbor of e , which in turn is neighbor of a implying that b is a neighbor of a .
4. Iterate through the remaining unvisited points in the dataset. Those points that do not belong to any cluster are noise.

DBSCAN Clustering Algorithm Solved Example

- | | | |
|----------------------------|--------------|------|
| • Apply the DBSCAN | Data Points: | |
| algorithm to the given | P1: (3, 7) | P2: |
| data points and | P3: (5, 5) | P4: |
| | P5: (7, 3) | P6: |
| • Create the clusters with | P7: (7, 2) | P8: |
| • minPts = 4 and | P9: (3, 3) | P10: |

DBSCAN Clustering Algorithm Solved Exam

- Use Euclidian distance and calculate the distance between points.

$$\text{Distance}(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

DBSCAN Clustering Algorithm Solved Exam

minPts = 4 and epsilon (ε) = 1.9

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11
P1	0										
P2	1.41	0									
P3	2.83	1.41	0								
P4	4.24	2.83	1.41	0							
P5	5.66	4.24	2.83	1.41	0						
P6	5.83	4.47	3.16	2.00	1.41	0					
P7	6.40	5.00	3.61	2.24	1.00	1.00	0				
P8	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0			
P9	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0		

DBSCAN Clustering Algorithm Solved Exam

minPts = 4 and epsilon (ε) = 1.9

P1: P2, P10
P2: P1, P3, P11
P3: P2, P4
P4: P3, P5
P5: P4, P6, P7, P8
P6: P5, P7
P7: P5, P6
P8: P5
P9: P12

Point	Status	
P1	Noise	Border
P2	Core	
P3	Noise	Border
P4	Noise	Border
P5	Core	
P6	Noise	Border
P7	Noise	Border
P8	Noise	Border
P9	Noise	

DBSCAN Clustering Algorithm Solved Example

P1: P2, P10

P2: P1, P3, P11

P3: P2, P4

P4: P3, P5

P5: P4, P6, P7, P8

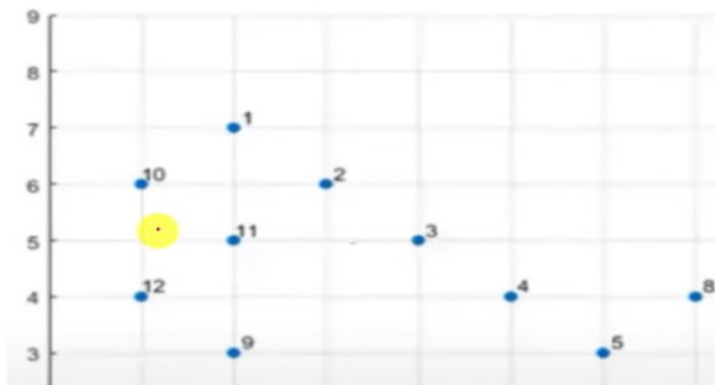
P6: P5, P7

P7: P5, P6

P8: P5

P9: P12

minPts = 4 and epsilon (ϵ) = 1.9



Minimum Spanning Tree (MST) based clustering is a class of graph-based clustering algorithms that leverages the properties of the MST to partition data. It's particularly effective for finding clusters with **arbitrary shapes** and for serving as a foundation for hierarchical clustering.

- **Graph Representation:** The dataset is first modeled as a **complete, weighted, undirected graph** $G=(V,E)$.
 - **Vertices (V):** Each data point is a vertex.
 - **Edges (E):** Edges connect every pair of vertices.
 - **Weights:** The weight of an edge between two points is their **distance** (e.g., Euclidean distance), representing their dissimilarity.
- **Minimum Spanning Tree (MST):** An MST is a subgraph of G that connects all vertices together with the **minimum possible total edge weight** and **no cycles**.
- **Clustering Idea:** In a well-separated dataset, the edges that connect different clusters (i.e., **inter-cluster edges**) are typically **much longer** than the edges connecting points

within the same cluster (i.e., *intra-cluster* edges). By **removing the longest/inconsistent edges** from the MST, the tree breaks into disconnected subtrees, where each subtree represents a cluster.

Algorithm Steps

1. **Construct the Complete Graph (G):** Compute the pairwise distance between all data points to create the weighted graph.
2. **Compute the MST:** Use an efficient algorithm like **Kruskal's** or **Prim's** to find the Minimum Spanning Tree (T).
 - *Note: Since the MST connects all data points via the shortest possible "path," it captures the intrinsic connectivity structure of the data.*
3. **Cluster Formation (Edge Cutting):**
 - **Identify Inconsistent Edges:** Determine which edges are "inconsistent" (significantly longer) compared to their neighbors in the tree. This is the crucial step that defines the specific MST clustering algorithm.
 - **Remove Edges:** Remove the inconsistent edges. Each remaining **connected component** (subtree) is a cluster.

Numerical Example for MST Clustering

Here is a step-by-step numerical example for MST clustering using a small set of 2D data points.

1. Initial Data and Distances

Consider five data points P1 to P5:

Point	Coordinates (x,y)
P1	(1, 1)
P2	(2, 1)
P3	(4, 3)
P4	(4, 4)
P5	(5, 4)

Calculate the **Euclidean distance** between all pairs of points. This forms the weighted graph where points are vertices and distances are edge weights.

Edge	Distance (Weight)
(P1,P2)	$(2-1)^2+(1-1)^2=1.0$
(P3,P4)	$(4-4)^2+(4-3)^2=1.0$
(P4,P5)	$(5-4)^2+(4-4)^2=1.0$
(P3,P5)	$(5-4)^2+(4-3)^2=1^2+1^2=2.0$
(P2,P3)	$(4-2)^2+(3-1)^2=2^2+2^2=8.0$
(P2,P4)	$(4-2)^2+(4-1)^2=2^2+3^2=13.0$
(P1,P3)	$(4-1)^2+(3-1)^2=3^2+2^2=13.0$
...	<i>(other edges are longer)</i>

2. Construct the Minimum Spanning Tree (MST)

Using an algorithm like **Kruskal's Algorithm**, we select edges in increasing order of weight, ensuring no cycles are formed. The MST will have $N-1=5-1=4$ edges.

Step	Edge Selected	Weight	Cycle?	Action
1	(P1,P2)	1.0	No	Add to MST
2	(P3,P4)	1.0	No	Add to MST
3	(P4,P5)	1.0	No	Add to MST
4	(P3,P5)	1.41	Yes (forms cycle P3–P4–P5–P3)	Reject
5	(P2,P3)	2.83	No (connects {P1,P2} to {P3,P4,P5})	Add to MST
-	(MST Complete)			

The **MST edges** are:

(P1,P2), (P3,P4), (P4,P5), and (P2,P3). The edge weights in the MST are: 1.0,1.0,1.0,2.83.

3. Clustering by Breaking MST Edges

To form clusters, we remove the longest edges from the MST. The removal of $K-1$ edges results in K clusters.

Forming $K=3$ Clusters

1. Start with the MST.
2. **Remove the longest edge** (1st longest): (P2,P3) (weight 2.83). Clusters: {P1,P2} and {P3,P4,P5}.
3. **Identify the next longest edge** in the remaining forest. The remaining edges are: (P1,P2),(P3,P4),(P4,P5), all with weight 1.0. Let's arbitrarily pick (P4,P5).
4. **Remove** (P4,P5) (weight 1.0).
5. The MST breaks into three connected components:
 - **Cluster 1:** {P1,P2}
 - **Cluster 2:** {P3,P4}
 - **Cluster 3:** {P5}